SỐ MẪU THI GIỮA KỲ · React Native + TS · Snack SDK 54

Bỏ tính năng = xoá dòng có nhãn: **//P phân trang** **//R kéo làm mới** **//A gọi API** **//S tìm** **//O sắp xếp**
① = trong App trước return · ② = chỗ nội dung JSX

Mẫu 0 – Khung App.tsx (dùng chung)

```
import React, { useState, useEffect, useCallback, useMemo, useRef }
  from 'react';
import { View, Text, TextInput, FlatList, ActivityIndicator,
  TouchableOpacity, RefreshControl, Modal, Alert, Platform, StyleSheet }
  from 'react-native';
import { SafeAreaProvider, SafeAreaView }
  from 'react-native-safe-area-context';
// [ngoài App: type, API, hook]
export default function App() {
  // ①
  return (
    <SafeAreaProvider><SafeAreaView style={s.safe}>
      <Text style={s.title}>Tiêu đề</Text>
      {/* ② */}
    </SafeAreaView></SafeAreaProvider>
  );
}
const s = StyleSheet.create({
  safe: { flex: 1, padding: 12, backgroundColor: 'whitesmoke' },
  title: { fontSize: 24, fontWeight: 'bold', textAlign: 'center' },
  center: { alignItems: 'center', gap: 12, marginTop: 40 },
  btn: { backgroundColor: 'tomato', padding: 10, borderRadius: 8 },
  row: { backgroundColor: 'white', padding: 12, borderRadius: 8 },
  chip: { padding: 8, borderRadius: 16, backgroundColor: 'white' },
  chipOn: { backgroundColor: 'royalblue' },
});
```

Mẫu 1 – Fetch danh sách: loading · lỗi + Thử lại · rỗng · //P · //R

Ngoài App (để không bắt tách file → để luôn trong App.tsx):

```
type Item = { id: string; name: string }; // types.ts
const API = 'https://xxx.mockapi.io/items'; // api.ts
const LIMIT = 6; // 0 = không phân trang
async function getPage(page: number): Promise<Item[]> {
  const url = LIMIT ? `${API}?page=${page}&limit=${LIMIT}` : API;
  const res = await fetch(url);
  if (!res.ok) throw new Error('HTTP ' + res.status);
  return res.json();
}
function useItems() { // hooks/useItems.ts
  const [items, setItems] = useState<Item[]>([]);
  const [loading, setLoading] = useState(true);
  const [refreshing, setRefreshing] = useState(false);
  const [loadingMore, setLoadingMore] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [page, setPage] = useState(1);
  const [hasMore, setHasMore] = useState(true);
  const load = useCallback(async (refresh = false) => {
    if (refresh) setRefreshing(true); else setLoading(true);
    setError(null);
    try {
      const data = await getPage(1);
      setItems(data); setPage(1);
      setHasMore(LIMIT > 0 && data.length === LIMIT);
    } catch (e) { setError('Không tải được dữ liệu.'); }
    finally { setLoading(false); setRefreshing(false); }
  }, []);
  const loadMore = useCallback(async () => {
    if (loadingMore || !hasMore) return;
    setLoadingMore(true);
    try {
      const data = await getPage(page + 1);
      setItems((prev) => [...prev,
        ...data.filter((d) => !prev.some((p) => p.id === d.id))]);
      setPage(page + 1); setHasMore(data.length === LIMIT);
    } catch (e) {} finally { setLoadingMore(false); }
  }, [page, hasMore, loadingMore]);
  useEffect(() => { load(); }, [load]);
  return { items, error, load, loadMore, loading, refreshing,
    loadingMore };
}
```

① Trong App (App.tsx):

```
const { items, error, load, loadMore, loading, refreshing, loadingMore }
  = useItems();
```

② JSX (App.tsx):

```
{loading ? <ActivityIndicator size="large" /> : error ? (
  <View style={s.center}>
    <Text style={{ color: 'red' }}>{error}</Text>
    <TouchableOpacity style={s.btn} onPress={() => load()}>
      <Text>Thử lại</Text></TouchableOpacity>
  </View>
) : (
  <FlatList data={items} keyExtractor={(i) => i.id}
    renderItem={({ item }) => <Text style={s.row}>{item.name}</Text>}
    ItemSeparatorComponent={() => <View style={{ height: 8 }} />}
    ListEmptyComponent={<Text>Không có dữ liệu.</Text>}
    onEndReached={loadMore} onEndReachedThreshold={0.5} //P
    ListFooterComponent={loadingMore ? <ActivityIndicator /> : null} //P
    refreshControl={<RefreshControl refreshing={refreshing} //R
      onRefresh={() => load(true)} />} //R
  />
)}
```

Mẫu 2 – FlatList: props · App.tsx (Đề 3–4: screens/…)

```
<FlatList data={items} keyExtractor={(item) => item.id}
 renderItem={({ item, index }) => <…/>} ItemSeparatorComponent={() =>
 <View style={{ height: 8 }} />} ListHeaderComponent={<…/>}
 ListEmptyComponent={<Text>Không có dữ liệu.</Text>}
 ListFooterComponent={loadingMore ? <ActivityIndicator /> : null}
 contentContainerStyle={{ padding: 12 }} onEndReached={loadMore}
 onEndReachedThreshold={0.5} refreshControl={<RefreshControl
 refreshing={refreshing} onRefresh={() => load(true)} />} | refreshing
 onRefresh />
```

Lưới: numColumns={2} key="2" columnWrapperStyle={{ gap: 8 }} (ô width 48%) · **Ngang:** horizontal (không chung numColumns)

Mẫu 3 – Modal form thêm mới (kiểu Câu 4 Đề 2) · App.tsx

State: [open, setOpen] = useState(false) · [title, setTitle] · [priority, setPriority] = useState<Priority>('medium') · inputRef = useRef<TextInput>(null)

<Modal visible={open} transparent animationType="slide" onShow={() => inputRef.current?.focus()} onRequestClose={() => setOpen(false)}**>** · nền View {flex: 1, justifyContent: 'flex-end'} · khung {backgroundColor: 'white', padding: 16, borderRadius: 16}

<TextInput ref={inputRef} value={title} onChangeText={setTitle} **/>** · chọn 1: PRIORITIES.map(p => chip onPress={() => setPriority(p)} style={[s.chip, priority === p && s.chipOn]})

Lưu: disabled={!title.trim()} style={[s.btn, !title.trim() && {backgroundColor: 'lightgray'}]} onPress={handleSave} = handleAdd (Mẫu 4) + setOpen(false); setTitle('') · dueDate: new Date().toISOString().slice(0, 10)

Đặt ở đâu: 1 Modal duy nhất, trong SafeAreaView, ngay sau FlatList (ngang hàng) – KHÔNG đặt trong renderItem

Xem chi tiết: [selected, setSelected] = useState<Item | null>(null) · visible={selected !== null} · {selected && (…)}

Mẫu 4 – CRUD axios (GET · POST · PUT · DELETE) + thêm/sửa/xoá state

Ngoài App:

```
import axios from 'axios'; // api.ts
import { Task, NewTask } from './types';
export const api = axios.create({ baseURL: 'https://xxx.mockapi.io' });
export const getTasks = async (): Promise<Task[]> =>
  (await api.get<Task[]>('/tasks')).data;
export const getTask = async (id: string): Promise<Task> =>
  (await api.get<Task>(`/tasks/${id}`)).data;
export const addTask = async (task: NewTask): Promise<Task> =>
  (await api.post<Task>('/tasks', task)).data;
export const updateTask = async (id: string, changes: Partial<Task>):
  Promise<Task> => (await api.put<Task>(`/tasks/${id}`, changes)).data;
export const deleteTask = async (id: string) =>
  api.delete(`/tasks/${id}`);
```

① Trong App (App.tsx):

```
const handleAdd = async (title: string) => {
  try {
    const created = await addTask({ title, done: false });
    setTasks((prev) => [created, ...prev]);
  } catch (e) { Alert.alert('Lỗi', 'Không thêm được.'); }
};
const handleToggle = useCallback(async (task: Task) => {
  try {
    await updateTask(task.id, { done: !task.done }); //A
    setTasks((prev) => prev.map((t) =>
      t.id === task.id ? { ...t, done: !t.done } : t));
  } catch (e) { Alert.alert('Lỗi', 'Không cập nhật được.'); }
}, []);
const handleDelete = useCallback((task: Task) => {
  const remove = async () => {
    try {
      await deleteTask(task.id); //A
      setTasks((prev) => prev.filter((t) => t.id !== task.id));
    } catch (e) { Alert.alert('Lỗi', 'Không xoá được.'); }
  };
  if (Platform.OS === 'web') {
    if (window.confirm(`Xoá "${task.title}"?`)) remove();
  } else Alert.alert('Xoá công việc?', task.title, [
    { text: 'Huỷ', style: 'cancel' },
    { text: 'Xoá', style: 'destructive', onPress: remove }, // không ()
  ]);
}, []);
```

Mẫu 5 – Lọc bằng chip + tìm (//S) + sắp xếp (//O) — useMemo

Ngoài App (App.tsx · ORDER có thể ở types.ts):

```
type Filter = 'all' | 'todo' | 'done';
const FILTERS: { key: Filter; label: string }[] = [
  { key: 'all', label: 'Tất cả' }, { key: 'todo', label: 'Chưa xong' },
  { key: 'done', label: 'Đã xong' },
];
const ORDER: Record<Priority, number> = { high: 0, medium: 1, low:
  2 };//O
```

① Trong App (App.tsx):

```
const [filter, setFilter] = useState<Filter>('all');
const [query, setQuery] = useState('');
const visible = useMemo(() => {
  let list = tasks.filter((t) =>
    filter === 'all' || (filter === 'done' ? t.done : !t.done));
  const q = query.trim().toLowerCase(); //S
  list = list.filter((t) => t.title.toLowerCase().includes(q)); //S
  list.sort((a, b) => ORDER[a.priority] - ORDER[b.priority]); //O
  return list;
}, [tasks, filter, query]);
// chip danh mục: c === 'Tất cả' || t.category === c
```

② JSX (App.tsx):

```
<TextInput value={query} onChangeText={setQuery} />{/*//S*/}
<View style={{ flexDirection: 'row', gap: 8 }}>
  {FILTERS.map((f) => (
    <TouchableOpacity key={f.key} onPress={() => setFilter(f.key)}
      style={[s.chip, filter === f.key && s.chipOn]}>
      <Text>{f.label}</Text>
    </TouchableOpacity>
  ))}
</View>
{/* FlatList data={visible} + ListEmptyComponent */}
```

Mẫu 6 – Stack + fetch API (Đề 3): Home (list) → Detail (id) → Cart

Chạy ngay trong App.tsx (import + styles của khung) · tách file: theo tên file cuối dòng:

```
import { Button, Image, ScrollView } from 'react-native';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator, NativeStackScreenProps }
  from '@react-navigation/native-stack';
type Product = { id: string; name: string; category: string; // types.ts
  price: number; rating: number; image: string; inStock: boolean };
type RootStackParamList = { Home: undefined;
  Detail: { productId: string }; Cart: undefined };
type P<T extends keyof RootStackParamList> =
  NativeStackScreenProps<RootStackParamList, T>;
const formatVnd = (n: number) => `${n.toLocaleString('vi-VN')} đ`;
const API_URL = // api.ts
  'https://6ad2f22366cd28aed82e56d5.mockapi.io/products';
async function getJson<T>(url: string): Promise<T> {
```

```
  const res = await fetch(url);
  if (!res.ok) throw new Error('HTTP ' + res.status);
  return (await res.json()) as T;
}
function HomeScreen({ navigation }: P<'Home'>) {
  const [error, setError] = useState(''); // screens/HomeScreen.tsx
  const [loading, setLoading] = useState(true);
  const [items, setItems] = useState<Product[]>([]);
  const load = async () => {
    setLoading(true); setError('');
    try { setItems(await getJson<Product[]>(API_URL)); }
    catch (e) { setError('Không tải được danh sách.'); }
    finally { setLoading(false); }
  };
  useEffect(() => { load(); }, []);
  if (loading) return <ActivityIndicator style={s.center} />;
  if (error) return (
    <View style={s.center}><Text style={{ color: 'red' }}>{error}</Text>
      <Button title="Thử lại" onPress={load} /></View>
  );
  return (
    <FlatList data={items} keyExtractor={(p) => p.id}
      contentContainerStyle={{ padding: 12, gap: 8 }}
      renderItem={({ item }) => (
        <TouchableOpacity style={[s.row, { flexDirection: 'row' }]}
          onPress={() => navigation.navigate('Detail',
            { productId: item.id })}>
          <Text style={{ flex: 1 }}>{item.name}</Text>
          <Text>{formatVnd(item.price)}</Text>
        </TouchableOpacity>
      )} />
  );
}
type DetailState = { status: 'loading' } | { status: 'error' }
  | { status: 'success'; data: Product };
function DetailScreen({ route, navigation }: P<'Detail'>) {
  const { productId } = route.params; // screens/DetailScreen.tsx
  const url = `${API_URL}/${productId}`;
  const [st, setSt] = useState<DetailState>({ status: 'loading' });
  useEffect(() => {
    let cancelled = false;
    getJson<Product>(url).then((data) => {
      if (cancelled) return;
      setSt({ status: 'success', data });
      navigation.setOptions({ title: data.name }); // đổi tiêu đề
    }).catch(() => { if (!cancelled) setSt({ status: 'error' }); });
    return () => { cancelled = true; }; // dọn dẹp: rời màn/id đổi
    // hẹn giờ: const t = setTimeout(…); return () => clearTimeout(t);
  }, [url, navigation]);
  if (st.status === 'loading') return <ActivityIndicator />;
  if (st.status === 'error') return <Text>Không tải được.</Text>;
  const p = st.data;
  const fields: [string, string][] = [['ID', p.id], ['Tên', p.name],
    ['Danh mục', p.category], ['Giá', formatVnd(p.price)],
    ['Đánh giá', String(p.rating)], ['Ảnh', p.image],
    ['Tình trạng', p.inStock ? 'Còn hàng' : 'Hết hàng']];
  return (
    <ScrollView contentContainerStyle={{ padding: 16, gap: 8 }}>
      {p.image.startsWith('http') && <Image source={{ uri: p.image }}
        style={{ width: '100%', aspectRatio: 1 }} />}
      <Text style={{ color: 'gray' }}>GET {url}</Text>
      {fields.map(([k, v]) => (
        <View key={k} style={[s.row, { flexDirection: 'row',
          justifyContent: 'space-between' }]}>
          <Text style={{ color: 'gray' }}>{k}</Text><Text>{v}</Text>
        </View>
      ))}
      <Button title="Tới giỏ"
        onPress={() => navigation.navigate('Cart')} />
      <Button title="Quay lại" onPress={() => navigation.goBack()} />
    </ScrollView>
  );
}
function CartScreen({ navigation }: P<'Cart'>) {
  return ( // screens/CartScreen.tsx
    <View style={s.center}><Text>Màn Cart</Text>
      <Button title="Về Home" onPress={() => navigation.popToTop()} />
    </View>
  );
}
const Stack = // App.tsx
  createNativeStackNavigator<RootStackParamList>();
export default function App() {
  return (
    <SafeAreaProvider><NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" component={HomeScreen}
          options={({ navigation }) => ({
            title: 'Sản phẩm',
            headerRight: () => <Button title="🛒"
              onPress={() => navigation.navigate('Cart')} />,
          })} />
        <Stack.Screen name="Detail" component={DetailScreen}
          options={{ title: 'Chi tiết' }} />
        <Stack.Screen name="Cart" component={CartScreen}
          options={{ title: 'Giỏ hàng' }} />
      </Stack.Navigator>
    </NavigationContainer></SafeAreaProvider>
  );
}
```

Tabs (Đề 4) — navigation/RootNavigator.tsx: import { createBottomTabNavigator } from '@react-navigation/bottom-tabs' · const Tab = createBottomTabNavigator<TabParamList>() (TabParamList ở types.ts) · <Tab.Navigator screenOptions={{ tabBarActiveTintColor: 'teal' }}> <Tab.Screen name="Notes" component={…} options={{ title, tabBarIcon: () => <Text>📝</Text> }} /> · lồng vào Stack: <Stack.Screen name="Main" component={MainTabs} options={{ headerShown: false }} />

Từ màn trong Tab mở màn Stack cha (screens/…): const navigation = useNavigation<NativeStackNavigationProp<RootStackParamList>>() · **Đăng nhập (navigation/RootNavigator.tsx):** {user ? <Stack.Screen name="Main" …/> : <Stack.Screen name="Login" …/>} (không navigate)

Mẫu 7 – Giỏ hàng: Context + useReducer (thêm · giảm · xoá · xoá hết)

Ngoài App:

```
// context/CartContext.tsx
import { createContext, useContext, useMemo, useReducer, ReactNode }
  from 'react'; import { Product } from '../types';
export interface CartItem { product: Product; qty: number }
export type CartAction =
  | { type: 'add'; product: Product }
  | { type: 'decrease'; id: string }
  | { type: 'remove'; id: string }
  | { type: 'clear' };
export function cartReducer(s: CartItem[], a: CartAction): CartItem[] {
  switch (a.type) {
    case 'add': {
      const id = a.product.id;
      if (!s.some((i) => i.product.id === id))
        return [...s, { product: a.product, qty: 1 }];
      return s.map((i) =>
        i.product.id === id ? { ...i, qty: i.qty + 1 } : i);
    }
    case 'decrease':
      return s.map((i) =>
        i.product.id === a.id ? { ...i, qty: i.qty - 1 } : i)
        .filter((i) => i.qty > 0);
    case 'remove':
      return s.filter((i) => i.product.id !== a.id);
    case 'clear':
      return [];
  }
}
interface CartValue {
  items: CartItem[]; count: number; total: number;
  add: (p: Product) => void; decrease: (id: string) => void;
  remove: (id: string) => void; clear: () => void;
}
const CartContext = createContext<CartValue | undefined>(undefined);
export function CartProvider({ children }: { children: ReactNode }) {
  const [items, dispatch] = useReducer(cartReducer, []);
  const count = useMemo(
    () => items.reduce((n, i) => n + i.qty, 0), [items]);
  const total = useMemo(
    () => items.reduce((n, i) => n + i.qty * i.product.price, 0),
    [items]);
  const value: CartValue = { items, count, total,
    add: (product) => dispatch({ type: 'add', product }),
    decrease: (id) => dispatch({ type: 'decrease', id }),
    remove: (id) => dispatch({ type: 'remove', id }),
    clear: () => dispatch({ type: 'clear' }) };
  return (
    <CartContext.Provider value={value}>{children}</CartContext.Provider>
  );
}
export function useCart() {
  const ctx = useContext(CartContext);
  if (!ctx) throw new Error('useCart phải nằm trong <CartProvider>');
  return ctx;
}
// App.tsx: <CartProvider> bọc ngoài <NavigationContainer>
```

① Dùng giỏ — import { useCart } from '../context/CartContext':

```
// components/CartButton.tsx (headerRight): số món trên nút
const { count } = useCart();   // <Text>{`🛒 ${count}`}</Text>
// screens/DetailScreen.tsx (gọi TRƯỚC các lệnh return sớm):
//   const { add } = useCart(); → onPress={() => add(p)}
// screens/CartScreen.tsx:
const { items, total, add, decrease, remove, clear } = useCart();
```

② JSX (screens/CartScreen.tsx):

```
<FlatList data={items} keyExtractor={(i) => i.product.id}
  ListEmptyComponent={<Text>Giỏ hàng trống.</Text>}
  renderItem={({ item }) => (
    <View style={{ flexDirection: 'row', alignItems: 'center', gap: 8 }}>
      <Text style={{ flex: 1 }}>{item.product.name}</Text>
      <TouchableOpacity onPress={() => decrease(item.product.id)}>
        <Text>−</Text></TouchableOpacity>
      <Text>{item.qty}</Text>
      <TouchableOpacity onPress={() => add(item.product)}>
        <Text>+</Text></TouchableOpacity>
      <Text>{formatVnd(item.qty * item.product.price)}</Text>
      <TouchableOpacity onPress={() => remove(item.product.id)}>
        <Text>Xoá</Text></TouchableOpacity>
    </View>
  )} />
<Text>{`Tổng: ${formatVnd(total)}`}</Text>
<TouchableOpacity disabled={items.length === 0} onPress={clear}>
  <Text>Xoá giỏ</Text></TouchableOpacity>
```

Mục Note

Omit (types.ts): type NewTask = Omit<Task, 'id'> → addTask(task: NewTask) (api.ts) · **Partial<Task>:** mọi trường tuỳ chọn → updateTask(id, { done: true })

Record (types.ts): const LABEL: Record<Priority, string> = { low: 'Thấp', medium: 'Vừa', high: 'Cao' }; → dùng: LABEL[task.priority]

Không cuộn: mọi View bọc FlatList phải có style={{ flex: 1 }} · nhiều chip: <ScrollView horizontal style={{ flexGrow: 0, flexShrink: 0 }}>

<Switch value={task.done} onValueChange={() => onToggle(task)} **/>** (components/TaskItem.tsx · không dùng onChange) · disabled · trackColor · thumbColor

Đề 3 – Add dependency: @react-navigation/native · @react-navigation/native-stack · react-native-screens · react-native-safe-area-context (Tabs: @react-navigation/bottom-tabs · API: axios)

Đề 3 – component đặc biệt: screens/HomeScreen.tsx: lưới numColumns={2} columnWrapperStyle={{ justifyContent: 'space-between' }} + ô width '48%' · ảnh vuông: <Image source={{ uri }} style={{ width: '100%', aspectRatio: 1 }} /> · nhãn "Hết hàng" đè góc: <View><Image/>{!inStock && <Text style={{ position: 'absolute', top: 6, right: 6 }}>…}</View> · screens/DetailScreen.tsx: màn dài <ScrollView contentContainerStyle={{ padding: 16, gap: 8 }}> · tên 1 dòng: numberOfLines={1} · báo: Platform.OS === 'web' ? window.alert(msg) : Alert.alert('Giỏ hàng', msg) · nút khoá: disabled={!p.inStock} style={[s.btn, !p.inStock && { backgroundColor: 'lightgray' }]}

Tách file: export (màn/component: export default) · import ở App.tsx: './types' './screens/HomeScreen' · ở screens/ components/ context/ hooks/: '../types' '../api'